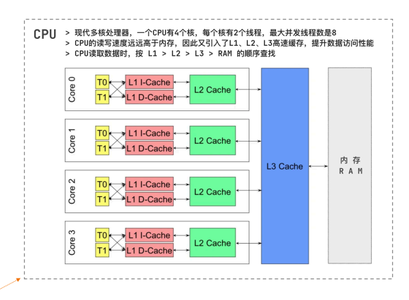
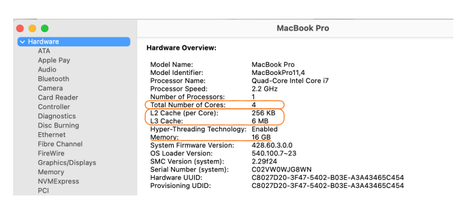

CPU
> 现代多核处理器，一个CPU有4个核，每个核有2个线程，最大并发线程数是8
> CPU的读写速度远远高于内存，因此又引入了L1、L2、L3高速缓存，提升数据访问性能
> CPU读取数据时，按 L1 > L2 > L3 > RAM 的顺序查找
Core 0
Core 1
Core 2
Core 3
T0
T1
L1 I-Cache
L1 D-Cache
L2 Cache
L3 Cache
内 存
R A M



Hardware Overview:

Model Name: MacBook Pro
Model Identifier: MacBookPro11,4
Processor Name: Quad-Core Intel Core i7
Processor Speed: 2.2 GHz
Number of Processors: 1
Total Number of Cores: 4
L2 Cache (per Core): 256 KB
L3 Cache: 6 MB
Hyper-Threading Technology: Enabled
Memory: 16 GB
System Firmware Version: 428.60.3.0.0
OS Loader Version: 540.100.7~23
SMC Version (system): 2.29f24
Serial Number (system): C02VW0WJG8WN
Hardware UUID: C8027D20-3F47-5402-B03E-A3A43465C454
Provisioning UDID: C8027D20-3F47-5402-B03E-A3A43465C454

JMM Java内存模型

1. 什么是内存模型?
   随着计算机硬件的升级发展，出现了CPU高速缓存、指令重排等优化技术，多线程并发环境下会出现共享内存访问问题
   例如，线程各自持有共享内存的缓存，优先更新自己缓存的数据，看不见其他线程的修改，导致数据不一致
   内存模型，定义了多线程并发访问共享内存时的行为规范，解决这些问题

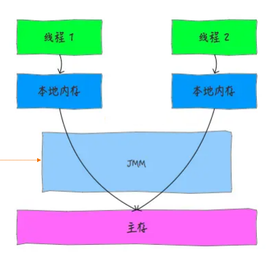
2. 硬件内存模型：CPU厂商提供了内存屏障指令，上层软件可使用这些指令：1)保证缓存一致 2)禁止重排序

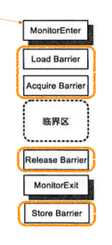
3. JMM：java版本的内存模型，屏蔽底层硬件和操作系统差异，保证java程序在各种平台都能正确访问共享内存

4. JMM实现：JVM抽象出一套内存屏障方法，在不同硬件上运行时，生成对应的内存屏障指令 1)保证缓存一致 2)禁止重排序
   封装这些内存屏障方法，提供一些关键字给开发者使用，例如：volatile、synchronized、final等

5. volatile：执行完写操作，调用storeload()内存屏障方法，执行lock指令：1) 将缓存行的内容立即写回内存，并使其他CPU缓存失效（多线程间的可见性）
   2) 禁止指令重排

6. synchronized: only one thread can enter a given critical section of the code at any given time
   when a thread enters a synchronized block, all variables visible to the thread are refreshed from main memory
   when a thread exits a synchronized block, all updated variables are flushed back to main memory





```
src/hotspot/share/interpreter/zero/bytecodeInterpreter.cpp::run() //解释器执行bytecode
→ switch(opcode) //读取opcode，执行相关操作
   CASE(_putfield): //存储实例field
   CASE(_putstatic): //存储类变量
     ...
     if (cache→is_volatile()) {
       switch (tos_type) { //根据变量类型调用相关方法复制
         case btos:obj→release_byte_field_put(field_offset, STACK_INT(-1)); break;
         case ctos:obj→release_char_field_put(field_offset, STACK_INT(-1)); break;
         ...
       }
       OrderAccess::storeload(); //写操作之后，执行lock指令，达到内存屏障效果
     } else {...}
   CASE(_getfield): //获取实例变量
   CASE(_getstatic): //获取类变量
     ...
     if (cache→is_volatile()) {
       if (support_IRIW_for_not_multiple_copy_atomic_cpu) { //绝大多数平台都是true
         OrderAccess::fence(); //读操作调用这个干吗?
       }
       switch (tos_type) {...} //读操作
     } else {...}
```

```
void oopDesc::release_byte_field_put(int offset, jbyte contents) {
  OrderAccess::release_store(byte_field_addr(offset), contents);
}
void OrderAccess::release_store(volatile jbyte* p, jbyte v) {
 *p = v;
}
```

```
void OrderAccess::storeload() → fence() {
  if (os::is_MP()) { //多核处理器
    /* 汇编指令，锁住当前缓存行: 1).将cache line内容立即写回内存，并使其他CPU的缓存失效（可见性）
     *                         2).禁止指令重排（有序性）   */
    #ifdef AMD64
      __asm__ volatile ("lock; addl $0,0(%%rsp)" : : : "cc", "memory");
    #else
      __asm__ volatile ("lock; addl $0,0(%%esp)" : : : "cc", "memory");
    #endif
  }
  //如果是单核处理器，不加内存屏障
}
```

```
//DCL双重校验锁单例
public class A {
  private static volatile A singleton;
  private A() {} //构造函数是private的，避免在其他地方创建单例

  public static A getSingleton() {
     if (singleton == null) { //1.第一次check：避免单例创建后继续进入同步块，造成不必要的损耗
        synchronized (A.class) { //2.lock：此时只有一个线程拿到锁，其他线程进入waitSet等锁
           if (singleton == null) { //3.2:第二次check：避免其他线程等到锁后重复创建单例
              singleton = new A(); //3.1:拿到锁的线程创建单例
           }
        }
     }
     return singleton; //返回单例
  }
}
```

```
//一行代码编译成了4条指令
(1) NEW A //分配实例
(2) DUP
(3) INVOKESPECIAL A.<init> ()V //初始化实例
(4) PUTSTATIC A.singleton : LA; //singleton指向实例
//volatile的内存屏障保证(4)不会被提前执行，避免了其他线程提前拿到没初始化的单例
```